```
/**
  ******************************************************************************
  * @file      startup_stm32f407vgtx.s
  * @author    Auto-generated by STM32CubeIDE
  * @Abstract : Startup script for STM32F407VGTx Device
  * @version   V1.0.0
  * @date      2020-02-14
  ******************************************************************************
  *
  * OVERVIEW FOR BEGINNERS:
  * ========================
  * This file is the very first code that runs when your STM32 microcontroller
  * powers on or resets. Before your C code (main()) can run, several things
  * must be set up at the hardware level. This file does those things:
  *
  *   1. Sets up the stack pointer (where function call data lives in RAM)
  *   2. Copies initialized global variables from Flash (permanent storage)
  *      into RAM (working memory), because RAM loses its contents on power loss
  *   3. Zeroes out uninitialized global variables (the BSS segment)
  *   4. Calls SystemInit() to configure clocks
  *   5. Calls __libc_init_array() to run C++ static constructors
  *   6. Jumps to your main() function
```

```
 *
24
 * It also defines the "vector table" — a lookup table of function pointers
25
 * that tells the hardware WHERE to jump when an interrupt or exception fires.
26
 *
27
 * This file is written in ARM Thumb assembly. You don't need to edit it
28
 * unless you are doing very low-level customization.
29
 */
30

31
/* ============================================================================
32
 * ASSEMBLER DIRECTIVES
33
 * These are instructions TO the assembler (the program that converts this
34
 * text into machine code), not instructions that run on the chip itself.
35
 * ============================================================================ */
36

37
.syntax unified      /* Use "unified" ARM/Thumb syntax — a modern syntax that
38
                        lets you write both ARM and Thumb instructions the
39
                        same way, instead of having two different styles. */
40

41
.cpu cortex-m4       /* Tell the assembler we're targeting the ARM Cortex-M4
42
                        core inside the STM32F407. This enables Cortex-M4
43
                        specific instructions. */
44

45
.fpu softvfp         /* "softvfp" = software floating point ABI with hardware
```

```
                      FPU registers. This means: use the hardware FPU unit
                      for calculations, but pass float arguments through
                      integer registers when calling functions.
                      The alternative is "hard" (hardfp) which passes floats
                      directly in FPU registers — slightly faster but
                      requires all libraries to be compiled the same way. */

.thumb              /* Generate Thumb-2 instructions (16/32-bit compact
                      encoding) rather than full 32-bit ARM instructions.
                      Cortex-M4 ONLY runs Thumb mode — it cannot execute
                      classic 32-bit ARM instructions at all. */

/* ============================================================================
 * GLOBAL SYMBOL EXPORTS
 * These lines make these two symbols visible to the linker and other files.
 * Without .global, a symbol is private to this file.
 * ============================================================================ */

.global g_pfnVectors   /* The interrupt vector table (defined later in this
                        file). The linker needs to find it to place it at
                        address 0x00000000 in Flash. */
```

```
.global Default_Handler /* A catch-all handler for any interrupt that doesn't
                           have a real implementation written yet. */

/* ============================================================================
 * LINKER SCRIPT SYMBOL REFERENCES
 * The .word directive places a 32-bit address literal into the output binary.
 * These symbols (starting with underscore) are NOT defined here — they are
 * defined in the linker script (.ld file) which knows the exact memory map
 * of this chip. The linker will fill in the actual addresses at link time.
 *
 * Think of this like declaring "extern" variables in C: we're telling the
 * assembler "these addresses exist somewhere, the linker will resolve them."
 * ============================================================================ */

.word _sidata  /* Source address in Flash where the initial values of
                  initialized global variables are stored.
                  ("si" = source of initialized data) */

.word _sdata   /* Start address in RAM where .data section begins.
                  ("s" = start, "data" = initialized globals/statics) */

.word _edata   /* End address in RAM where .data section ends.
                  ("e" = end) */
```

```
91

92
.word _sbss    /* Start address in RAM of the .bss section.
93
                  BSS = Block Started by Symbol — this is where uninitialized
94
                  global and static variables live. C standard says they must
95
                  start as zero, so we'll zero this region out explicitly. */
96

97
.word _ebss    /* End address in RAM of the .bss section. */
98

99

100
/* ============================================================================
101
 * RESET HANDLER
102
 * ============================================================================
103
 *
104
 * This is the entry point of your entire program. When the chip powers on
105
 * or is reset, the hardware automatically:
106
 *   1. Reads address 0x00000000 → loads it into the Stack Pointer (SP)
107
 *   2. Reads address 0x00000004 → loads it into the Program Counter (PC)
108
 *
109
 * Address 0x00000004 in the vector table contains the address of
110
 * Reset_Handler, so execution jumps here immediately after reset.
111
 *
112
 * .section .text.Reset_Handler
113
```

```
 *   Put this code in a special named section so the linker can place it
114
 *   correctly. The linker script will pull this into the .text region of Flash.
115
 *
116
 * .weak Reset_Handler
117
 *   "Weak" means: if another file defines a symbol with this same name,
118
 *   use that one instead and discard this one. This lets application code
119
 *   override the default startup behavior if needed.
120
 *
121
 * .type Reset_Handler, %function
122
 *   Tells the assembler (and debugger) that this symbol is a function,
123
 *   not just a data label. Important for correct stack unwinding in debuggers.
124
 */
125

126
  .section .text.Reset_Handler
127
  .weak Reset_Handler
128
  .type Reset_Handler, %function
129

130
Reset_Handler:
131

132
  /* --- Step 1: Initialize the Stack Pointer ---
133
   *
134
   * The stack is a region of RAM used to store:
135
   *   - Return addresses when calling functions
```

```
136
   *   - Local variables
137
   *   - Saved register values during interrupts
138
   *
139
   * On Cortex-M, the stack grows DOWNWARD in memory (from high address to low).
140
   * _estack is defined in the linker script as the TOP of RAM (highest address).
141
   * We load this address into SP (Stack Pointer register, also called r13).
142
   *
143
   * "ldr r0, =_estack" is a pseudo-instruction. The assembler generates a
144
   * PC-relative load from a literal pool — it's loading the 32-bit address
145
   * value of _estack into register r0.
146
   *
147
   * "mov sp, r0" copies that value into the actual Stack Pointer register.
148
   *
149
   * After this, any function call or interrupt will use this stack correctly.
150
   */
151
  ldr   r0, =_estack
152
  mov   sp, r0
153

154


155
  /* --- Step 2: Copy .data section from Flash to RAM ---
156
   *
157
   * WHY IS THIS NECESSARY?
158
```

```
 * Flash memory retains its contents when power is removed, but it's
159
 * read-only at runtime (you can't write to Flash during normal execution
160
 * without a special erase/write procedure). RAM is fast and writable,
161
 * but loses everything on power-off.
162
 *
163
 * When you write C code like:
164
 *     int myGlobal = 42;   // initialized global variable
165
 *
166
 * The value 42 must be stored somewhere permanently (Flash), but the
167
 * variable itself must live in RAM so it can be modified. The linker puts
168
 * the initial value (42) into a Flash region called .sidata, and reserves
169
 * a slot in RAM called .data. This startup code copies from .sidata → .data
170
 * before main() runs, so your globals have their correct initial values.
171
 *
172
 * The copy loop below:
173
 *   r0 = destination start (RAM: _sdata)
174
 *   r1 = destination end   (RAM: _edata)
175
 *   r2 = source start      (Flash: _sidata)
176
 *   r3 = byte offset counter (starts at 0, increments by 4 each iteration)
177
 *
178
 * We jump to LoopCopyDataInit first to check whether there's anything
179
 * to copy at all (handles the edge case of an empty .data section).
180
 */
```

```
181
  ldr r0, =_sdata    /* Load address of start of .data section in RAM */
182
  ldr r1, =_edata    /* Load address of end of .data section in RAM */
183
  ldr r2, =_sidata   /* Load address of initial values stored in Flash */
184
  movs r3, #0        /* r3 = 0 (byte offset, starts at beginning) */
185
  b LoopCopyDataInit /* Jump to the loop condition check first */
186

187
CopyDataInit:
188
  /* Loop body: copy one 32-bit word (4 bytes) per iteration */
189
  ldr r4, [r2, r3]   /* Load 4 bytes from Flash: address = r2 + r3 offset */
190
  str r4, [r0, r3]   /* Store those 4 bytes to RAM:  address = r0 + r3 offset */
191
  adds r3, r3, #4    /* Advance offset by 4 bytes (one word) */
192
                     /* "adds" sets condition flags (N, Z, C, V) — needed by bcc below */
193

194
LoopCopyDataInit:
195
  /* Loop condition: check if we've reached the end of .data */
196
  adds r4, r0, r3    /* r4 = current destination address (RAM base + offset) */
197
  cmp r4, r1         /* Compare current address against end-of-.data (_edata) */
198
  bcc CopyDataInit   /* BCC = Branch if Carry Clear = Branch if r4 < r1
199
                        i.e., loop back if we haven't reached the end yet.
200
                        When r4 >= r1, we fall through and the copy is done. */
201

202

203
```

```
  /* --- Step 3: Zero-fill the .bss section in RAM ---
204
   *
205
   * WHY IS THIS NECESSARY?
206
   * The C standard guarantees that uninitialized global and static variables
207
   * start at zero. For example:
208
   *     int counter;           // uninitialized global — must start as 0
209
   *     static int flag;       // uninitialized static — must start as 0
210
   *
211
   * The linker places these in a section called .bss (historically "Block
212
   * Started by Symbol"). Nothing is stored in Flash for .bss (there are no
213
   * initial values to store — they're all zero). We just need to zero out
214
   * that RAM region ourselves before main() runs.
215
   *
216
   * r2 = current address being zeroed (starts at _sbss)
217
   * r4 = end address (_ebss)
218
   * r3 = the value 0 (what we're writing)
219
   *
220
   * Same pattern as above: jump to condition check first.
221
   */
222
  ldr r2, =_sbss     /* Load start address of .bss in RAM */
223
  ldr r4, =_ebss     /* Load end address of .bss in RAM */
224
  movs r3, #0        /* r3 = 0 (the value to write everywhere) */
225
  b LoopFillZerobss  /* Jump to condition check first */
```

```
226

227
FillZerobss:
228
  /* Loop body: write one 32-bit zero per iteration */
229
  str  r3, [r2]      /* Store 0 into current address (r2 points to RAM) */
230
  adds r2, r2, #4    /* Advance pointer by 4 bytes */
231

232
LoopFillZerobss:
233
  /* Loop condition: have we zeroed everything up to _ebss? */
234
  cmp r2, r4         /* Compare current pointer against end of .bss */
235
  bcc FillZerobss    /* Loop back if r2 < r4 (not done yet) */
236

237

238
  /* --- Step 4: Initialize the system clock ---
239
   *
240
   * SystemInit() is a function provided by ST's CMSIS/HAL library (in
241
   * system_stm32f4xx.c). It configures the PLL (Phase-Locked Loop) and
242
   * clock prescalers to run the chip at its target frequency (up to 168 MHz
243
   * on the F407). Without this call, the chip runs from the slow internal
244
   * RC oscillator at 16 MHz with no PLL.
245
   *
246
   * "bl" = Branch with Link. It calls the function AND saves the return
247
   * address in the Link Register (LR / r14), so the function can return
248
```

```
   * here with "bx lr".
249
   */
250
  bl  SystemInit
251


252


253
  /* --- Step 5: Run C++ static constructors ---
254
   *
255
   * __libc_init_array() is part of the C runtime library (newlib).
256
   * It iterates over the .init_array section in Flash and calls every
257
   * function pointer stored there. The compiler places constructors for
258
   * global C++ objects there, as well as any functions marked with
259
   * __attribute__((constructor)) in C.
260
   *
261
   * If you're writing pure C with no global objects, this effectively does
262
   * nothing — but it's safe and correct to always call it.
263
   */
264
  bl __libc_init_array
265


266


267
  /* --- Step 6: Call main() ---
268
   *
269
   * Everything is now set up. Jump to the user's application entry point.
270
   * "bl main" is technically a function call (saves return address in LR),
```

```
271
   * which is correct — main() could theoretically return, though on a
272
   * bare-metal embedded system it normally never does.
273
   */
274
  bl main
275


276


277
  /* --- Step 7: Trap if main() ever returns ---
278
   *
279
   * On an embedded system, main() returning is almost always a bug —
280
   * there's nowhere sensible to go back to. This infinite loop prevents
281
   * the program counter from wandering into random memory and executing
282
   * garbage instructions, which could cause unpredictable hardware behavior.
283
   *
284
   * A debugger can detect the chip is stuck here by examining the PC register.
285
   */
286
LoopForever:
287
    b LoopForever     /* "b" = unconditional Branch — jumps back to itself forever */
288


289


290
  /* Tell the assembler the size of Reset_Handler (used by debuggers for
291
     stack unwinding and function boundary detection).
292
     "." means "current address", so this computes: current_addr - Reset_Handler_start */
293
```

```
  .size Reset_Handler, .-Reset_Handler
294
295
296
/* ================================================================================
297
 * DEFAULT INTERRUPT HANDLER
298
 * ================================================================================
299
 *
300
 * If an interrupt fires but no specific handler function has been written
301
 * for it, the weak alias mechanism (defined at the bottom of this file)
302
 * redirects it here. This handler just spins forever in an infinite loop.
303
 *
304
 * In practice, if you hit this handler unexpectedly, it means:
305
 *   - An interrupt was enabled that you forgot to write a handler for
306
 *   - An interrupt fired due to a hardware misconfiguration or a bug
307
 *
308
 * When debugging, you can set a breakpoint here to catch unexpected interrupts.
309
 * Then examine the stack and LR register to figure out which interrupt fired.
310
 *
311
 * "ax" in the .section directive means:
312
 *   a = allocatable (takes up space in the binary)
313
 *   x = executable (contains instructions, not just data)
314
 * %progbits means it contains actual program data (not zero-fill like .bss)
315
 */
```

```
316
  .section .text.Default_Handler,"ax",%progbits
317
Default_Handler:
318
Infinite_Loop:
319
  b Infinite_Loop    /* Spin here forever. The system is in an unknown state.
320
                        A watchdog timer (if configured) will eventually reset
321
                        the chip and give it another chance to run correctly. */
322
  .size Default_Handler, .-Default_Handler
323

324

325
/* ===========================================================================
326
 * INTERRUPT VECTOR TABLE
327
 * ===========================================================================
328
 *
329
 * WHAT IS THE VECTOR TABLE?
330
 * On ARM Cortex-M, when an interrupt or exception occurs, the hardware
331
 * automatically looks up a table of function pointers stored at the very
332
 * beginning of Flash memory (address 0x00000000). Each slot in the table
333
 * corresponds to a specific exception or interrupt source. The hardware reads
334
 * the address stored in that slot and jumps to it — this is the "handler".
335
 *
336
 * The table MUST be placed at address 0x00000000 (or at a location pointed
337
 * to by the VTOR register — Vector Table Offset Register — if you move it).
338
```

```
 * The linker script uses the ".isr_vector" section name to ensure this.
339
 *
340
 * TABLE LAYOUT:
341
 * Slot 0:  Initial Stack Pointer value (not a function — it's loaded into SP at reset)
342
 * Slot 1:  Reset Handler address (where execution begins)
343
 * Slot 2+: Exception and interrupt handler addresses
344
 *
345
 * Each entry is a 32-bit address (.word = 4 bytes).
346
 * Entries that say "0" are "reserved" by ST — those interrupt numbers don't
347
 * exist on this chip, so the slot is left empty (null pointer).
348
 *
349
 * "a" in .section means allocatable (included in the binary output).
350
 * %progbits means it contains real data.
351
 */
352
  .section .isr_vector,"a",%progbits
353
  .type g_pfnVectors, %object
354
  .size g_pfnVectors, .-g_pfnVectors
355

356
g_pfnVectors:
357

358
  /* ---- ARM Cortex-M Core Exceptions (positions 0–15) ---- */
359
  /* These are defined by the ARM architecture itself, not ST.
360
     Every Cortex-M chip has these same first 16 entries. */
```

```
361
362
  .word _estack                /* [0] Initial stack pointer value.
363
                                  NOT a handler address — the hardware loads
364
                                  this directly into SP (r13) on reset.
365
                                  _estack is the top of RAM. */
366
367
  .word Reset_Handler          /* [1] Reset exception — runs on power-on/reset.
368
                                  This is the entry point defined above. */
369
370
  .word NMI_Handler            /* [2] Non-Maskable Interrupt — cannot be
371
                                  disabled by software. Used for critical
372
                                  faults like clock failure. */
373
374
  .word HardFault_Handler      /* [3] Hard Fault — fires when a fault occurs
375
                                  that has no more specific handler, or when
376
                                  a fault occurs inside another fault handler.
377
                                  Common causes: null pointer dereference,
378
                                  stack overflow, bad instruction. */
379
380
  .word MemManage_Handler      /* [4] Memory Management Fault — MPU (Memory
381
                                  Protection Unit) access violation. Fires
382
                                  when code tries to access memory it's not
383
```

```
                                allowed to (e.g., executing from RAM when
384
                                the MPU forbids it). */
385

386
  .word BusFault_Handler       /* [5] Bus Fault — error on the AHB/APB bus
387
                                during a memory access. Can be caused by
388
                                accessing a non-existent peripheral address
389
                                or bad DMA configuration. */
390

391
  .word UsageFault_Handler     /* [6] Usage Fault — undefined instruction,
392
                                illegal unaligned access, invalid state
393
                                transition (e.g., trying to switch to
394
                                ARM mode from Thumb-only Cortex-M). */
395

396
  .word 0                      /* [7] Reserved by ARM spec */
397
  .word 0                      /* [8] Reserved by ARM spec */
398
  .word 0                      /* [9] Reserved by ARM spec */
399
  .word 0                      /* [10] Reserved by ARM spec */
400

401
  .word SVC_Handler            /* [11] SVCall (Supervisor Call) — triggered by
402
                                the "svc" instruction. Used by RTOS kernels
403
                                (like FreeRTOS) to request privileged
404
                                operations from unprivileged task code. */
405
```

```
406
  .word DebugMon_Handler       /* [12] Debug Monitor — fires when a software
407
                                  breakpoint or watchpoint is hit while the
408
                                  debug monitor is active (as opposed to
409
                                  being halted by a hardware debugger). */
410
411
  .word 0                      /* [13] Reserved by ARM spec */
412
413
  .word PendSV_Handler         /* [14] PendSV (Pendable Service Call) — a
414
                                  software-triggerable, low-priority interrupt.
415
                                  FreeRTOS and other RTOSes use this for
416
                                  context switching between tasks, because it
417
                                  can be pended (deferred) and will run after
418
                                  all higher-priority interrupts complete. */
419
420
  .word SysTick_Handler        /* [15] SysTick — a simple 24-bit countdown
421
                                  timer built into the Cortex-M core.
422
                                  HAL uses this for the HAL_Delay() 1ms tick.
423
                                  FreeRTOS uses it for the scheduler tick. */
424
425
426
  /* ---- STM32F407-Specific Peripheral Interrupts (positions 16+) ----
427
   *
428
```

```
   * Everything from here on is specific to the STM32F407 chip design by ST.
429
   * These are IRQs (Interrupt Requests) from the on-chip peripherals.
430
   * The number in the comment is the IRQ number (position - 16).
431
   * In C code you reference these with enum values like USART1_IRQn.
432
   *
433
   * In your application code, you define a handler by simply writing a
434
   * function with the exact same name, e.g.:
435
   *
436
   *     void USART1_IRQHandler(void) {
437
   *         // handle received byte
438
   *     }
439
   *
440
   * Because these are declared .weak below, your version overrides the
441
   * default (which just spins forever in Default_Handler).
442
   */
443

444
  .word WWDG_IRQHandler        /* IRQ0:  Window Watchdog — a timer that resets
445
                                  the chip if your firmware "forgets" to pet
446
                                  it. Window = must be refreshed within a
447
                                  specific time window, not too early or late. */
448

449
  .word PVD_IRQHandler         /* IRQ1:  PVD (Programmable Voltage Detector) —
450
                                  fires when VDD supply voltage drops below
```

```
451
                                a programmable threshold. Useful for
452
                                detecting a dying battery before it causes
453
                                data corruption. */
454
455
  .word TAMP_STAMP_IRQHandler  /* IRQ2:  Tamper detection + Timestamp via EXTI.
456
                                RTC tamper pins detect physical tampering
457
                                with the hardware. */
458
459
  .word RTC_WKUP_IRQHandler    /* IRQ3:  RTC (Real-Time Clock) Wakeup — wakes
460
                                the chip from Stop/Standby low-power modes
461
                                on a scheduled interval. */
462
463
  .word 0                      /* IRQ4:  Reserved (not used on STM32F407) */
464
465
  .word RCC_IRQHandler         /* IRQ5:  RCC (Reset and Clock Control) —
466
                                clock security system interrupt. Fires if
467
                                the external crystal oscillator fails so
468
                                software can switch to the internal backup
469
                                oscillator. */
470
471
  .word EXTI0_IRQHandler       /* IRQ6:  EXTI Line 0 — External Interrupt on
472
                                GPIO pin 0 (PA0, PB0, PC0, etc.).
473
```

```
                                    Used to detect button presses, encoder
474
                                    pulses, or any digital signal edge. */
475

476
  .word EXTI1_IRQHandler       /* IRQ7:  EXTI Line 1 – GPIO pin 1 */
477
  .word EXTI2_IRQHandler       /* IRQ8:  EXTI Line 2 – GPIO pin 2 */
478
  .word EXTI3_IRQHandler       /* IRQ9:  EXTI Line 3 – GPIO pin 3 */
479
  .word EXTI4_IRQHandler       /* IRQ10: EXTI Line 4 – GPIO pin 4 */
480

481
  .word DMA1_Stream0_IRQHandler /* IRQ11: DMA1 Stream 0.
482
                                    DMA = Direct Memory Access. The DMA
483
                                    controller moves data between peripherals
484
                                    and RAM without involving the CPU, freeing
485
                                    the CPU to do other work. Each "stream"
486
                                    is an independent DMA channel.
487
                                    This fires when the transfer completes,
488
                                    or when a transfer error occurs. */
489

490
  .word DMA1_Stream1_IRQHandler /* IRQ12: DMA1 Stream 1 */
491
  .word DMA1_Stream2_IRQHandler /* IRQ13: DMA1 Stream 2 */
492
  .word DMA1_Stream3_IRQHandler /* IRQ14: DMA1 Stream 3 */
493
  .word DMA1_Stream4_IRQHandler /* IRQ15: DMA1 Stream 4 */
494
  .word DMA1_Stream5_IRQHandler /* IRQ16: DMA1 Stream 5 */
495
  .word DMA1_Stream6_IRQHandler /* IRQ17: DMA1 Stream 6 */
```

```
496
497
  .word ADC_IRQHandler         /* IRQ18: ADC (Analog-to-Digital Converter)
498
                                  global interrupt. The F407 has 3 ADCs.
499
                                  Fires on conversion complete, overrun,
500
                                  or injected channel end. */
501
502
  .word CAN1_TX_IRQHandler     /* IRQ19: CAN1 Transmit — CAN (Controller Area
503
                                  Network) is a robust serial protocol used
504
                                  in automotive and industrial systems.
505
                                  This fires when a CAN frame finishes
506
                                  transmitting (mailbox empty). */
507
508
  .word CAN1_RX0_IRQHandler    /* IRQ20: CAN1 Receive FIFO 0 — fires when a
509
                                  CAN frame is received into FIFO 0. */
510
511
  .word CAN1_RX1_IRQHandler    /* IRQ21: CAN1 Receive FIFO 1 */
512
513
  .word CAN1_SCE_IRQHandler    /* IRQ22: CAN1 Status Change / Error —
514
                                  fires on bus-off, error passive, or
515
                                  last error code changes. */
516
517
  .word EXTI9_5_IRQHandler     /* IRQ23: EXTI Lines 5–9 — GPIO pins 5,6,7,8,9
518
```

```
                                share a single interrupt. Your handler must
519
                                check the EXTI_PR register to determine
520
                                which specific pin triggered it. */
521

522
  .word TIM1_BRK_TIM9_IRQHandler  /* IRQ24: TIM1 Break + TIM9 global.
523
                                TIM1 is an "advanced" timer with PWM
524
                                features. The Break input shuts down PWM
525
                                outputs on a fault condition (e.g., overcurrent).
526
                                TIM9 is a general-purpose timer. */
527

528
  .word TIM1_UP_TIM10_IRQHandler  /* IRQ25: TIM1 Update (counter overflow/underflow)
529
                                + TIM10 global interrupt. */
530

531
  .word TIM1_TRG_COM_TIM11_IRQHandler /* IRQ26: TIM1 Trigger & Commutation
532
                                (used in 3-phase motor control) + TIM11 global. */
533

534
  .word TIM1_CC_IRQHandler     /* IRQ27: TIM1 Capture/Compare — fires when
535
                                a timer channel captures an input signal
536
                                timestamp or matches an output compare value. */
537

538
  .word TIM2_IRQHandler        /* IRQ28: TIM2 global — 32-bit general purpose
539
                                timer, useful for precise time measurement. */
540
```

```
541
  .word TIM3_IRQHandler        /* IRQ29: TIM3 global — 16-bit general purpose timer */
542
  .word TIM4_IRQHandler        /* IRQ30: TIM4 global — 16-bit general purpose timer */
543

544
  .word I2C1_EV_IRQHandler     /* IRQ31: I2C1 Event — I2C is a 2-wire serial
545
                                  bus (SDA + SCL) used for sensors, EEPROMs,
546
                                  displays, etc. The Event interrupt fires on
547
                                  address match, byte received/sent, STOP detected. */
548

549
  .word I2C1_ER_IRQHandler     /* IRQ32: I2C1 Error — bus error, arbitration
550
                                  loss, NACK received, etc. */
551

552
  .word I2C2_EV_IRQHandler     /* IRQ33: I2C2 Event */
553
  .word I2C2_ER_IRQHandler     /* IRQ34: I2C2 Error */
554

555
  .word SPI1_IRQHandler        /* IRQ35: SPI1 global — SPI (Serial Peripheral
556
                                  Interface) is a fast 4-wire bus (MOSI, MISO,
557
                                  SCK, CS) used for SD cards, displays,
558
                                  ADCs, sensors, etc. */
559

560
  .word SPI2_IRQHandler        /* IRQ36: SPI2 global */
561

562
  .word USART1_IRQHandler      /* IRQ37: USART1 — Universal Synchronous/
563
```

```
                                Asynchronous Receiver-Transmitter. This is
564
                                your standard serial/UART. Fires on byte
565
                                received, transmit buffer empty, etc.
566
                                Commonly used for debug output or talking
567
                                to a PC over USB-serial. */
568

569
  .word USART2_IRQHandler      /* IRQ38: USART2 global */
570
  .word USART3_IRQHandler      /* IRQ39: USART3 global */
571

572
  .word EXTI15_10_IRQHandler   /* IRQ40: EXTI Lines 10–15 — GPIO pins 10–15.
573
                                Same shared-interrupt situation as EXTI9_5. */
574

575
  .word RTC_Alarm_IRQHandler   /* IRQ41: RTC Alarm A or B through EXTI —
576
                                Wake up / trigger at a specific date/time. */
577

578
  .word OTG_FS_WKUP_IRQHandler /* IRQ42: USB OTG Full-Speed Wakeup — wakes
579
                                the chip from low-power mode when USB
580
                                activity is detected. */
581

582
  .word TIM8_BRK_TIM12_IRQHandler  /* IRQ43: TIM8 Break + TIM12 global */
583
  .word TIM8_UP_TIM13_IRQHandler   /* IRQ44: TIM8 Update + TIM13 global */
584
  .word TIM8_TRG_COM_TIM14_IRQHandler /* IRQ45: TIM8 Trigger/Commutation + TIM14 */
585
  .word TIM8_CC_IRQHandler     /* IRQ46: TIM8 Capture/Compare */
```

```
586
  .word DMA1_Stream7_IRQHandler /* IRQ47: DMA1 Stream 7 */
587
588
  .word FSMC_IRQHandler        /* IRQ48: FSMC (Flexible Static Memory
589
                                  Controller) — used to interface with
590
                                  external parallel memories like SRAM,
591
                                  NOR Flash, and LCD controllers. */
592
593
  .word SDIO_IRQHandler        /* IRQ49: SDIO — SD card interface interrupt.
594
                                  Fires on transfer complete, FIFO errors, etc. */
595
596
  .word TIM5_IRQHandler        /* IRQ50: TIM5 global — 32-bit general purpose timer */
597
  .word SPI3_IRQHandler        /* IRQ51: SPI3 global */
598
599
  .word UART4_IRQHandler       /* IRQ52: UART4 global — Note: UART (not USART).
600
                                  UARTs lack the synchronous clock line that
601
                                  USARTs have. Still async serial. */
602
603
  .word UART5_IRQHandler       /* IRQ53: UART5 global */
604
605
  .word TIM6_DAC_IRQHandler    /* IRQ54: TIM6 global + DAC underrun error.
606
                                  TIM6 is often used as the trigger source
607
                                  for the DAC (Digital-to-Analog Converter).
608
```

```
                                    Underrun = DAC ran out of data to output. */
609
610
  .word TIM7_IRQHandler        /* IRQ55: TIM7 global — basic timer, often used
611
                                    as a timebase for DAC or periodic callbacks */
612
613
  .word DMA2_Stream0_IRQHandler /* IRQ56: DMA2 Stream 0 */
614
  .word DMA2_Stream1_IRQHandler /* IRQ57: DMA2 Stream 1 */
615
  .word DMA2_Stream2_IRQHandler /* IRQ58: DMA2 Stream 2 */
616
  .word DMA2_Stream3_IRQHandler /* IRQ59: DMA2 Stream 3 */
617
  .word DMA2_Stream4_IRQHandler /* IRQ60: DMA2 Stream 4 */
618
619
  .word ETH_IRQHandler         /* IRQ61: Ethernet MAC global interrupt —
620
                                    the STM32F407 has a built-in 10/100 Mbps
621
                                    Ethernet MAC (Media Access Controller).
622
                                    Fires on frame received/sent, etc. */
623
624
  .word ETH_WKUP_IRQHandler    /* IRQ62: Ethernet Wakeup through EXTI —
625
                                    wake from low-power mode on network activity */
626
627
  .word CAN2_TX_IRQHandler     /* IRQ63: CAN2 Transmit */
628
  .word CAN2_RX0_IRQHandler    /* IRQ64: CAN2 Receive FIFO 0 */
629
  .word CAN2_RX1_IRQHandler    /* IRQ65: CAN2 Receive FIFO 1 */
630
  .word CAN2_SCE_IRQHandler    /* IRQ66: CAN2 Status/Error */
```

```
631
632
  .word OTG_FS_IRQHandler      /* IRQ67: USB OTG Full-Speed global interrupt –
633
                                  the STM32F407 has a built-in USB 2.0
634
                                  Full-Speed (12 Mbps) OTG controller.
635
                                  OTG = On-The-Go (can be host or device). */
636
637
  .word DMA2_Stream5_IRQHandler /* IRQ68: DMA2 Stream 5 */
638
  .word DMA2_Stream6_IRQHandler /* IRQ69: DMA2 Stream 6 */
639
  .word DMA2_Stream7_IRQHandler /* IRQ70: DMA2 Stream 7 */
640
641
  .word USART6_IRQHandler      /* IRQ71: USART6 global */
642
643
  .word I2C3_EV_IRQHandler     /* IRQ72: I2C3 Event */
644
  .word I2C3_ER_IRQHandler     /* IRQ73: I2C3 Error */
645
646
  .word OTG_HS_EP1_OUT_IRQHandler /* IRQ74: USB OTG High-Speed Endpoint 1 OUT –
647
                                  the F407 also has a USB HS (480 Mbps)
648
                                  controller (though it typically needs an
649
                                  external HS PHY chip). This handles data
650
                                  flowing OUT from host to device on EP1. */
651
652
  .word OTG_HS_EP1_IN_IRQHandler  /* IRQ75: USB OTG HS Endpoint 1 IN –
653
```

```
                                data flowing IN from device to host on EP1. */
654
655
  .word OTG_HS_WKUP_IRQHandler /* IRQ76: USB OTG HS Wakeup through EXTI */
656
  .word OTG_HS_IRQHandler      /* IRQ77: USB OTG HS global interrupt */
657
658
  .word DCMI_IRQHandler        /* IRQ78: DCMI (Digital Camera Interface) —
659
                                  parallel camera interface for connecting
660
                                  OV7670-style camera modules directly. */
661
662
  .word CRYP_IRQHandler        /* IRQ79: Cryptographic processor global —
663
                                  hardware AES, DES, Triple-DES acceleration. */
664
665
  .word HASH_RNG_IRQHandler    /* IRQ80: Hash (SHA-1, MD5) + RNG (Random Number
666
                                  Generator) global interrupt. The hardware
667
                                  RNG uses analog noise to generate true
668
                                  random numbers — useful for cryptography. */
669
670
  .word FPU_IRQHandler         /* IRQ81: FPU (Floating-Point Unit) global —
671
                                  fires on FPU exceptions like divide-by-zero,
672
                                  overflow, underflow, or invalid operation,
673
                                  if those exception traps are enabled. */
674
675
  /* Slots 82–87: Reserved by ST (these IRQ numbers don't exist on F407) */
```

```
676
  .word 0
677
  .word 0
678
  .word 0
679
  .word 0
680
  .word 0
681
  .word 0
682

683
  .word LCD_TFT_IRQHandler     /* IRQ88: LTDC (LCD-TFT Display Controller) —
684
                                  the F407 has a hardware layer compositor
685
                                  for driving RGB TFT displays. This fires
686
                                  on frame sync, line events, etc. */
687

688
  .word LCD_TFT_1_IRQHandler   /* IRQ89: LTDC Error interrupt — fires on FIFO
689
                                  underrun or transfer error in the display
690
                                  controller. */
691

692

693
/* ============================================================================
694
 * WEAK ALIASES FOR ALL INTERRUPT HANDLERS
695
 * ============================================================================
696
 *
697
 * WHAT IS A WEAK ALIAS?
698
```

```
 * The directives below do two things for each handler:
699
 *
700
 *   .weak SomeHandler
701
 *     Makes SomeHandler a "weak" symbol. If your application code defines
702
 *     a function with the exact same name, the linker uses YOUR version and
703
 *     ignores this one. If you DON'T define it, the linker keeps this one.
704
 *
705
 *   .thumb_set SomeHandler, Default_Handler
706
 *     Makes SomeHandler an alias (another name) for Default_Handler.
707
 *     Combined with .weak, this means: "by default, SomeHandler points to
708
 *     Default_Handler (the infinite loop), but any user-defined function
709
 *     called SomeHandler will override this."
710
 *
711
 * THE PRACTICAL RESULT:
712
 * You only need to write handlers for interrupts you actually use.
713
 * All others silently redirect to the infinite loop, making debugging easier
714
 * (you can set a breakpoint in Default_Handler to catch any unhandled IRQ).
715
 *
716
 * EXAMPLE:
717
 * If you want to handle UART1 bytes:
718
 *
719
 *   void USART1_IRQHandler(void) {     // exact name match required!
720
 *       char c = USART1->DR;           // read the received byte
```

```
721
 *       my_buffer_push(c);
722
 *   }
723
 *
724
 * The linker sees your strong definition and uses it instead of the
725
 * weak alias to Default_Handler below.
726
 */
727

728
  /* Core ARM exception handlers */
729
    .weak    NMI_Handler
730
    .thumb_set NMI_Handler,Default_Handler
731

732
    .weak    HardFault_Handler
733
    .thumb_set HardFault_Handler,Default_Handler
734

735
    .weak    MemManage_Handler
736
    .thumb_set MemManage_Handler,Default_Handler
737

738
    .weak    BusFault_Handler
739
    .thumb_set BusFault_Handler,Default_Handler
740

741
    .weak    UsageFault_Handler
742
    .thumb_set UsageFault_Handler,Default_Handler
743
```

```
744
    .weak    SVC_Handler
745
    .thumb_set SVC_Handler,Default_Handler
746
747
    .weak    DebugMon_Handler
748
    .thumb_set DebugMon_Handler,Default_Handler
749
750
    .weak    PendSV_Handler
751
    .thumb_set PendSV_Handler,Default_Handler
752
753
    .weak    SysTick_Handler
754
    .thumb_set SysTick_Handler,Default_Handler
755
756
    /* STM32F407 peripheral interrupt handlers */
757
    .weak    WWDG_IRQHandler
758
    .thumb_set WWDG_IRQHandler,Default_Handler
759
760
    .weak    PVD_IRQHandler
761
    .thumb_set PVD_IRQHandler,Default_Handler
762
763
    .weak    TAMP_STAMP_IRQHandler
764
    .thumb_set TAMP_STAMP_IRQHandler,Default_Handler
765
```

```
766
    .weak    RTC_WKUP_IRQHandler
767
    .thumb_set RTC_WKUP_IRQHandler,Default_Handler
768

769
    .weak    RCC_IRQHandler
770
    .thumb_set RCC_IRQHandler,Default_Handler
771

772
    .weak    EXTI0_IRQHandler
773
    .thumb_set EXTI0_IRQHandler,Default_Handler
774

775
    .weak    EXTI1_IRQHandler
776
    .thumb_set EXTI1_IRQHandler,Default_Handler
777

778
    .weak    EXTI2_IRQHandler
779
    .thumb_set EXTI2_IRQHandler,Default_Handler
780

781
    .weak    EXTI3_IRQHandler
782
    .thumb_set EXTI3_IRQHandler,Default_Handler
783

784
    .weak    EXTI4_IRQHandler
785
    .thumb_set EXTI4_IRQHandler,Default_Handler
786

787
    .weak    DMA1_Stream0_IRQHandler
788
```

```
.thumb_set DMA1_Stream0_IRQHandler,Default_Handler
789
790
.weak    DMA1_Stream1_IRQHandler
791
.thumb_set DMA1_Stream1_IRQHandler,Default_Handler
792
793
.weak    DMA1_Stream2_IRQHandler
794
.thumb_set DMA1_Stream2_IRQHandler,Default_Handler
795
796
.weak    DMA1_Stream3_IRQHandler
797
.thumb_set DMA1_Stream3_IRQHandler,Default_Handler
798
799
.weak    DMA1_Stream4_IRQHandler
800
.thumb_set DMA1_Stream4_IRQHandler,Default_Handler
801
802
.weak    DMA1_Stream5_IRQHandler
803
.thumb_set DMA1_Stream5_IRQHandler,Default_Handler
804
805
.weak    DMA1_Stream6_IRQHandler
806
.thumb_set DMA1_Stream6_IRQHandler,Default_Handler
807
808
.weak    ADC_IRQHandler
809
.thumb_set ADC_IRQHandler,Default_Handler
810
```

```
811
    .weak    CAN1_TX_IRQHandler
812
    .thumb_set CAN1_TX_IRQHandler,Default_Handler
813

814
    .weak    CAN1_RX0_IRQHandler
815
    .thumb_set CAN1_RX0_IRQHandler,Default_Handler
816

817
    .weak    CAN1_RX1_IRQHandler
818
    .thumb_set CAN1_RX1_IRQHandler,Default_Handler
819

820
    .weak    CAN1_SCE_IRQHandler
821
    .thumb_set CAN1_SCE_IRQHandler,Default_Handler
822

823
    .weak    EXTI9_5_IRQHandler
824
    .thumb_set EXTI9_5_IRQHandler,Default_Handler
825

826
    .weak    TIM1_BRK_TIM9_IRQHandler
827
    .thumb_set TIM1_BRK_TIM9_IRQHandler,Default_Handler
828

829
    .weak    TIM1_UP_TIM10_IRQHandler
830
    .thumb_set TIM1_UP_TIM10_IRQHandler,Default_Handler
831

832
    .weak    TIM1_TRG_COM_TIM11_IRQHandler
833
```

```
.thumb_set TIM1_TRG_COM_TIM11_IRQHandler,Default_Handler
834
835
.weak    TIM1_CC_IRQHandler
836
.thumb_set TIM1_CC_IRQHandler,Default_Handler
837
838
.weak    TIM2_IRQHandler
839
.thumb_set TIM2_IRQHandler,Default_Handler
840
841
.weak    TIM3_IRQHandler
842
.thumb_set TIM3_IRQHandler,Default_Handler
843
844
.weak    TIM4_IRQHandler
845
.thumb_set TIM4_IRQHandler,Default_Handler
846
847
.weak    I2C1_EV_IRQHandler
848
.thumb_set I2C1_EV_IRQHandler,Default_Handler
849
850
.weak    I2C1_ER_IRQHandler
851
.thumb_set I2C1_ER_IRQHandler,Default_Handler
852
853
.weak    I2C2_EV_IRQHandler
854
.thumb_set I2C2_EV_IRQHandler,Default_Handler
855
```

```
856
    .weak    I2C2_ER_IRQHandler
857
    .thumb_set I2C2_ER_IRQHandler,Default_Handler
858

859
    .weak    SPI1_IRQHandler
860
    .thumb_set SPI1_IRQHandler,Default_Handler
861

862
    .weak    SPI2_IRQHandler
863
    .thumb_set SPI2_IRQHandler,Default_Handler
864

865
    .weak    USART1_IRQHandler
866
    .thumb_set USART1_IRQHandler,Default_Handler
867

868
    .weak    USART2_IRQHandler
869
    .thumb_set USART2_IRQHandler,Default_Handler
870

871
    .weak    USART3_IRQHandler
872
    .thumb_set USART3_IRQHandler,Default_Handler
873

874
    .weak    EXTI15_10_IRQHandler
875
    .thumb_set EXTI15_10_IRQHandler,Default_Handler
876

877
    .weak    RTC_Alarm_IRQHandler
878
```

```
.thumb_set RTC_Alarm_IRQHandler,Default_Handler
879
880
.weak    OTG_FS_WKUP_IRQHandler
881
.thumb_set OTG_FS_WKUP_IRQHandler,Default_Handler
882
883
.weak    TIM8_BRK_TIM12_IRQHandler
884
.thumb_set TIM8_BRK_TIM12_IRQHandler,Default_Handler
885
886
.weak    TIM8_UP_TIM13_IRQHandler
887
.thumb_set TIM8_UP_TIM13_IRQHandler,Default_Handler
888
889
.weak    TIM8_TRG_COM_TIM14_IRQHandler
890
.thumb_set TIM8_TRG_COM_TIM14_IRQHandler,Default_Handler
891
892
.weak    TIM8_CC_IRQHandler
893
.thumb_set TIM8_CC_IRQHandler,Default_Handler
894
895
.weak    DMA1_Stream7_IRQHandler
896
.thumb_set DMA1_Stream7_IRQHandler,Default_Handler
897
898
.weak    FSMC_IRQHandler
899
.thumb_set FSMC_IRQHandler,Default_Handler
900
```

```
901
    .weak    SDIO_IRQHandler
902
    .thumb_set SDIO_IRQHandler,Default_Handler
903
904
    .weak    TIM5_IRQHandler
905
    .thumb_set TIM5_IRQHandler,Default_Handler
906
907
    .weak    SPI3_IRQHandler
908
    .thumb_set SPI3_IRQHandler,Default_Handler
909
910
    .weak    UART4_IRQHandler
911
    .thumb_set UART4_IRQHandler,Default_Handler
912
913
    .weak    UART5_IRQHandler
914
    .thumb_set UART5_IRQHandler,Default_Handler
915
916
    .weak    TIM6_DAC_IRQHandler
917
    .thumb_set TIM6_DAC_IRQHandler,Default_Handler
918
919
    .weak    TIM7_IRQHandler
920
    .thumb_set TIM7_IRQHandler,Default_Handler
921
922
    .weak    DMA2_Stream0_IRQHandler
923
```

```
.thumb_set DMA2_Stream0_IRQHandler,Default_Handler
924
925
.weak    DMA2_Stream1_IRQHandler
926
.thumb_set DMA2_Stream1_IRQHandler,Default_Handler
927
928
.weak    DMA2_Stream2_IRQHandler
929
.thumb_set DMA2_Stream2_IRQHandler,Default_Handler
930
931
.weak    DMA2_Stream3_IRQHandler
932
.thumb_set DMA2_Stream3_IRQHandler,Default_Handler
933
934
.weak    DMA2_Stream4_IRQHandler
935
.thumb_set DMA2_Stream4_IRQHandler,Default_Handler
936
937
.weak    ETH_IRQHandler
938
.thumb_set ETH_IRQHandler,Default_Handler
939
940
.weak    ETH_WKUP_IRQHandler
941
.thumb_set ETH_WKUP_IRQHandler,Default_Handler
942
943
.weak    CAN2_TX_IRQHandler
944
.thumb_set CAN2_TX_IRQHandler,Default_Handler
945
```

```
946
    .weak    CAN2_RX0_IRQHandler
947
    .thumb_set CAN2_RX0_IRQHandler,Default_Handler
948

949
    .weak    CAN2_RX1_IRQHandler
950
    .thumb_set CAN2_RX1_IRQHandler,Default_Handler
951

952
    .weak    CAN2_SCE_IRQHandler
953
    .thumb_set CAN2_SCE_IRQHandler,Default_Handler
954

955
    .weak    OTG_FS_IRQHandler
956
    .thumb_set OTG_FS_IRQHandler,Default_Handler
957

958
    .weak    DMA2_Stream5_IRQHandler
959
    .thumb_set DMA2_Stream5_IRQHandler,Default_Handler
960

961
    .weak    DMA2_Stream6_IRQHandler
962
    .thumb_set DMA2_Stream6_IRQHandler,Default_Handler
963

964
    .weak    DMA2_Stream7_IRQHandler
965
    .thumb_set DMA2_Stream7_IRQHandler,Default_Handler
966

967
    .weak    USART6_IRQHandler
968
```

```
.thumb_set USART6_IRQHandler,Default_Handler
969
970
.weak    I2C3_EV_IRQHandler
971
.thumb_set I2C3_EV_IRQHandler,Default_Handler
972
973
.weak    I2C3_ER_IRQHandler
974
.thumb_set I2C3_ER_IRQHandler,Default_Handler
975
976
.weak    OTG_HS_EP1_OUT_IRQHandler
977
.thumb_set OTG_HS_EP1_OUT_IRQHandler,Default_Handler
978
979
.weak    OTG_HS_EP1_IN_IRQHandler
980
.thumb_set OTG_HS_EP1_IN_IRQHandler,Default_Handler
981
982
.weak    OTG_HS_WKUP_IRQHandler
983
.thumb_set OTG_HS_WKUP_IRQHandler,Default_Handler
984
985
.weak    OTG_HS_IRQHandler
986
.thumb_set OTG_HS_IRQHandler,Default_Handler
987
988
.weak    DCMI_IRQHandler
989
.thumb_set DCMI_IRQHandler,Default_Handler
990
```

```
991
    .weak    CRYP_IRQHandler
992
    .thumb_set CRYP_IRQHandler,Default_Handler
993

994
    .weak    HASH_RNG_IRQHandler
995
    .thumb_set HASH_RNG_IRQHandler,Default_Handler
996

997
    .weak    FPU_IRQHandler
998
    .thumb_set FPU_IRQHandler,Default_Handler
999

1000
    .weak    LCD_TFT_IRQHandler
1001
    .thumb_set LCD_TFT_IRQHandler,Default_Handler
1002

1003
    .weak    LCD_TFT_1_IRQHandler
1004
    .thumb_set LCD_TFT_1_IRQHandler,Default_Handler
1005

1006
    /* SystemInit is also declared weak here. This means if you don't provide
1007
       your own SystemInit() (e.g., if you're not using ST's HAL/CMSIS), the
1008
       linker won't complain — it just won't call anything for clock setup.
1009
       Normally you DO want SystemInit() from system_stm32f4xx.c to run. */
1010
    .weak    SystemInit
1011

1012
/************************ (C) COPYRIGHT STMicroelectonics *****END OF FILE****/
1013
```